Semester Project: SkyNet Air Traffic Control (ATC) Simulator

Subject: Data Structures

Topic: Air Traffic Control Simulation

Group Members: Maximum 3 Students

Format: Console-Based C++ Application

1. Introduction

In this project, students will implement **SkyNet**, a console-based Air Traffic Control (ATC) simulator. The system will simulate a busy airspace where aircraft must be managed efficiently to prevent collisions, handle emergencies, and ensure timely landings.

The core challenge is to manage high-stakes data in real-time using **manually implemented data structures**. You cannot use standard libraries (STL) for the core components. The system must feature a visual "Radar" grid on the console to represent the airspace.

2. General Instructions

1. **Language:** C++ (Standard C++11 or higher).
2. **No STL:** Usage of `std::vector`, `std::priority_queue`, `std::map`, `std::stack`, or `std::queue` is **strictly prohibited** for the primary data structures. You must implement these classes yourself.
3. **Visuals:** The project is console-based. Use `system("cls")` or `system("clear")` to refresh the screen for the "Radar" view.
4. **Team Size:** Maximum of 3 students per group.
5. **Submission:** Submit all `.cpp` and `.h` files in a zip folder named `RollNumbers_SkyNetProject.zip`.

---

3. Module Specifications

Module A: The Airspace (Graph & 2D Grid)

- **Purpose:** Represent the flight paths and current positions of aircraft.
- **Data Structure: Weighted Directed Graph** (Adjacency List).
  - **Nodes:** Airports (e.g., JFK, LHR) and Waypoints (Checkpoints in the sky).
  - **Edges:** Flight corridors. **Weight** = Distance (in km) or Fuel Cost.
- **Console Visualization (The Radar):**
  - Implement a 20x20 (or larger) 2D array of `char`.

    - Display this grid every time the menu refreshes.
    - Use symbols: `A` for Airports, `.` for Empty Sky, `P` for Planes.
- **Requirements:**

    1. **Shortest Path:** Implement **Dijkstra's Algorithm** to calculate the safest route for a plane to the nearest airport in case of an emergency.
    2. **Movement:** Updates to the plane's location in the Graph must reflect on the 2D Radar grid.

Visual: A 2D Character Array printed to the console. **What's happening:** The system prints a grid where `.` is empty space, `A` is an Airport, and `P` represents planes.

```
================ SKYNET RADAR ================
   0 1 2 3 4 5 6 7 8 9
0 [A . . . . . . . . .]  <-- Airport (JFK)
1 [. . P1. . . . . . .]  <-- Plane PK-786
2 [. . . . . . . . P2.]  <-- Plane UA-909
3 [. . . . . . . . . .]
4 [. . . . . . A . . .]  <-- Airport (LHR)
==============================================
> Plane P1 is at Coordinates (1, 2)
> Plane P2 is at Coordinates (2, 8)
```

Module B: Landing Priority System (Min-Heap)

- **Purpose:** The runway is a scarce resource. Planes must land based on **Urgency**, not "First-Come-First-Served."
- **Data Structure: Min-Heap** (Manual Array-based implementation).
- **Priority Logic:**
    - **Priority 1 (CRITICAL):** Medical Emergency, Engine Failure, Hijack.
    - **Priority 2 (HIGH):** Low Fuel (< 10%).
    - **Priority 3 (MEDIUM):** Commercial Passenger Flight (Scheduled).
    - **Priority 4 (LOW):** Cargo / Private Jets.
- **Requirements:**

    1. **Request Landing:** Insert a flight into the Heap.
    2. **Process Landing:** `ExtractMin()` removes the highest priority plane from the sky and "lands" it (removes from Radar).
    3. **Emergency Override:** Ability to change a plane's priority mid-flight (e.g., a normal flight suddenly declares low fuel) using `DecreaseKey` logic.

Visual: A list showing who lands next. **What's happening:** You implemented a **Min-Heap**. Even though "Flight UA-909" arrived earlier, "Flight PK-786" is shown *first* because it has a higher urgency (Lower priority number = High Priority).

```
============= LANDING QUEUE (Min-Heap) =============
* NEXT TO LAND: Flight PK-786 [STATUS: CRITICAL] *
----------------------------------------------------
2. Flight UA-909 [Status: Low Fuel]
3. Flight BA-101 [Status: Normal]
4. Flight QR-555 [Status: Normal]

> Command: extract-min
> LANDING Flight PK-786... SUCCESS.
```

Module C: Aircraft Registry (Hash Table)

- **Purpose:** Instant access to aircraft details using its unique Flight ID (e.g., "PK-786").
- **Data Structure: Hash Table** with **Chaining** (Linked Lists) for collision handling.
- **Key:** Flight ID (String).
- **Value:** Aircraft Object (Model, Fuel Level, Origin, Destination, Current Status).
- **Requirements:**
    1. **Search:** $O(1)$ lookup to view details of any plane visible on the Radar.
    2. **Update:** Modify status (e.g., update Fuel Level or position) efficiently.

Visual: Instant lookup result. **What's happening:** The user types a Flight ID. The system uses a Hashing Function (e.g., SumOfASCII % TableSize) to find the data in $O(1)$ time, avoiding a slow linear search.

```
> Enter Flight ID: PK-786

SEARCHING DATABASE... FOUND.
---------------------------------
| Flight ID : PK-786            |
| Model     : Boeing 777        |
| Origin    : Lahore (LHE)      |
| Fuel      : 05% (CRITICAL)    |
| Status    : LANDED            |
---------------------------------
```

Module D: Flight Log History (AVL Tree)

- **Purpose:** A legal record of all flights that have successfully landed or crashed.
- **Data Structure: AVL Tree** (Self-Balancing Binary Search Tree).
- **Ordering:** Sort nodes by **Arrival Timestamp**.
- **Requirements:**
    1. **Insertion:** When a plane leaves the Heap (lands), insert a log record into the AVL Tree.

2. **Balance:** The tree must remain balanced (implement Rotations: LL, RR, LR, RL) to ensure $O(\log n)$ search speeds for historical data.
3. **Reporting:** Perform an **In-Order Traversal** to print the full daily report of landed flights chronologically.

Visual: A chronological list of the day's activity. **What's happening:** You perform an **In-Order Traversal** on your AVL Tree. Since AVL trees are sorted binary trees, an in-order traversal automatically prints the timestamps in perfect chronological order.

```
============= DAILY FLIGHT LOG =============
[08:00 AM] Flight ER-111 -> Landed safely.
[08:15 AM] Flight PK-786 -> EMERGENCY LANDING.
[09:30 AM] Flight UA-909 -> Landed safely.
[10:45 AM] Flight BA-101 -> Landed safely.
============================================
> End of Report.
```

Module E: Collision Avoidance (Collision Check)

- **Purpose:** Ensure two planes do not occupy the same "sector" (Grid coordinate/Graph Node) at the same time.
- **Logic:** Before moving a plane to a new node, check if the node is occupied. If occupied, the system must alert the user or hold the plane.

Visual: A text path connecting nodes. **What's happening:** The user asks for a route. The system calculates the shortest path through the Graph nodes and prints the sequence.

```
> Calculate Safe Route for: P2 (at Waypoint X)

CALCULATING PATH TO NEAREST AIRPORT...
--------------------------------------
Start: Waypoint X
   |
   V
Waypoint Y (Distance: 10km)
   |
   V
Waypoint Z (Distance: 5km)
   |
   V
Destination: Airport LHR (Total Dist: 25km)
--------------------------------------
```

---

4. System Features & Menu

The application must be menu-driven with the following main options:

1. **Airspace Visualizer:**
   - Displays the current "Radar" Grid.
   - Lists the Head of the Landing Queue (Next plane to land).
2. **Flight Control:**

- Add Flight: Spawns a new plane at an entry node.
- Declare Emergency: Updates a specific flight's status to Priority 1 (updates Heap).
- Land Flight: Processes the top of the Heap.

3. **Search & Reporting:**
   - Search Flight: Inputs Flight ID -> Displays info from Hash Table.
   - Print Log: Displays In-Order traversal of the AVL Tree.
   - Find Safe Route: Inputs current location -> Runs Dijkstra -> Displays path to nearest airport.
4. **System:**
   - Save/Load: Save the state of the Hash Table and AVL Tree to a .txt file (File I/O) so data persists after closing.